

labwork 6

October 3, 2026

0.1 Binarization exercise

In this first exercise the goal is to take an image and for each pixels ether put its value to 255 or to 0 depending on a threshold defined arbitrarily.

To do that we start by making a grayscale, then we take the value of the gray if it is higher than the threshold we put it to 255 if it is not we put it to 0.

```
@cuda.jit
def binarization(src, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y > src.shape[0] or x > src.shape[1]:
        return
    g = np.uint8((src[y, x, 0] + src[y, x, 1] + src[y, x, 2]) / 3)
    if (g > T):
        dst[y, x, 0] = dst[y, x, 1] = dst[y, x, 2] = 0
    else:
        dst[y, x, 0] = dst[y, x, 1] = dst[y, x, 2] = 255
```

0.2 Brightness exercise

In this second exercise, the goal is to increase the brightness of the image by a defined value.

To do that, for each values of RGB we add the defined value. To avoid exceeding 255 or 0 (if negative) we use the min function to take the minimal value between the source value and 255 and we use the max function to take the maximal value between the source value and 0.

```
@cuda.jit
def brightness(src, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y > src.shape[0] or x > src.shape[1]:
        return
    dst[y, x, 0] = max(min((src[y, x, 0] + V), 255), 0)
    dst[y, x, 1] = max(min((src[y, x, 1] + V), 255), 0)
    dst[y, x, 2] = max(min((src[y, x, 2] + V), 255), 0)
```

0.3 Blending exercise

In this last exercise, we want to mix images of the same size together. To do that, we multiply the source value of the first image to a given coefficient and add it to the opposite of the same coefficient multiplied by the source value of the second image.

```
@cuda.jit
def blend(src1, src2, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y > src1.shape[0] or x > src1.shape[1] or src1.shape[0] != src2.shape[0]:
        return
    dst[y, x, 0] = C * src1[y, x, 0] + (1 - C) * src2[y, x, 0]
    dst[y, x, 1] = C * src1[y, x, 1] + (1 - C) * src2[y, x, 1]
    dst[y, x, 2] = C * src1[y, x, 2] + (1 - C) * src2[y, x, 2]
```

0.4 Results

The original image:



